i4Twins
No-Code Platform For Industry 4.0 Solutions

Technical Task — Backend Engineer

Dana Tadbir Integrated Intelligent Systems Co.

Document code	Version	Date	Classification
DT-BE-TASK-01	**1.0**	**July 2026**	**For candidates**

1) Problem statement

You will design and implement a small backend service that ingests numeric sensor readings from a file, cleans duplicate and invalid records, and exposes time-based aggregation of the data through an API.

This task assesses your ability to analyze real messy data, your programming logic, software design and architecture, code cleanliness, and adherence to professional standards.

The suggested time for this task is at most two working days. If you estimate it will take longer, please let us know before proceeding.

2) What you will build

A service that reads the readings.jsonl file (one JSON reading per line) and satisfies the following requirements:

Expected deliverables

- **Ingestion:** read the readings from the readings.jsonl file.
- **Storage:** persist the cleaned readings in a data store of your choice (in-memory, SQLite, or another option); state the reason for your choice in the README.
- **Aggregation API:** a GET endpoint that, for a given device and metric within a specified time range, returns aggregated values per time bucket (contract details follow).
- **Count report:** at the end of processing, provide a summary of the total number of lines, readings stored, duplicates removed, and invalid records rejected. Also use an appropriate logging mechanism for significant events (start and end of processing, data errors, and the like).
- **Tests:** write at least unit tests for deduplication and for aggregation.
- **README:** including run and test instructions, a description of the key decisions and trade-offs in your own words, and disclosure of where and how AI was used (if used).

Design and architecture expectations

- **Domain logic independent of infrastructure:** the project structure should keep the domain logic independent of infrastructure details (file, database, and API). The choice of architectural style (Clean Architecture, Layered, Vertical Slice, and the like) is yours, but explain the reason in the README.
- **Domain-Driven Design:** design the domain model so that business rules (such as removing duplicate records and validating data) live in the domain layer, not in the Controller or the Repository.

- **Extensibility:** the design should make it possible to add a new aggregation type (such as Sum, Median, or Percentile) or a new input source (such as a message broker or HTTP) without widespread changes to the code.
- **Reasonable performance:** the chosen solution is expected to be reasonable in terms of time complexity and memory usage; explain any trade-offs in the README.

Robustness against messy data (the core of the task)

- **Safe deduplication (idempotency):** if a reading arrives twice, it must be stored only once.
- **Handling out-of-order data:** note that the file is not sorted by time.
- Rejecting and counting invalid records without crashing the service.

Structure of each reading

Field	Type	Example
deviceId	string	PUMP-01
metric	string	temperature / pressure / vibration
ts	string (ISO-8601 UTC)	2025-06-01T08:33:00Z
value	number	67.21
seq	integer	1199

Reading identity (for deduplication)

Two readings are considered “identical” when their (deviceId, metric, ts, seq) quadruple is equal. Design your own policy for handling duplicates and justify it in the README.

Aggregation API contract

- **Inputs:** deviceId, metric, the range start time (from), the range end time (to), and the bucket size (for example, in seconds).
- **Output:** for each equally sized time bucket within the range [from, to): the bucket start time, and the count, average, minimum, and maximum of the values for that device and metric that fall within the bucket. Only readings within the range must be included.
- The behavior for empty buckets (omitting them or returning them with a count of zero) is your choice; simply document it. The exact route shape and parameter names are up to you; what matters is the correctness of the service's behavior.

Important note: the input data is deliberately “messy” and includes duplicate records, out-of-order data, and invalid records. Discovering these in the data, choosing an appropriate policy for each, and documenting it in the README are part of the task.

3) Scope

- No real message queue, broker, or Kubernetes is required; reading the data directly from the file is sufficient.
- Authentication, a user interface, and production deployment are not required.
- Focus on: ingestion, cleaning and deduplication, aggregation, tests, the count report, and the README.
- C#/.NET 8 is our preference. If you use another language, state your reason and keep the code idiomatic and clean.

4) Submission terms

- Deliver the code in a Git repository with an incremental, meaningful commit history — not as a single final commit.
- Using AI tools is allowed, provided that you clearly state where and how they were used in the README.
- A short online session of a few minutes may be held, in which you walk us through your code.
- Final deliverable: the Git repository address along with the README file.

5) Evaluation criteria

The following matter to us in the evaluation:

- **Robustness against messy data:** correct deduplication and proper handling of invalid and out-of-order data.
- Correctness of the aggregation logic.
- **Code cleanliness and professional standards:** naming, structure, error handling, and readability.
- Quality and coverage of the tests.
- **Software design and architecture:** proper separation of concerns, domain logic independent of infrastructure, extensibility, and adherence to design principles (such as SOLID where appropriate).
- Clarity of the decisions in the README and a clear explanation in the possible online session.

6) What you are given

- The readings.jsonl file, containing roughly 2,150 readings from several devices and several metrics, sent to you along with this task.

We wish you every success.